

RL for Wordle
CS-5410: Reinforcement Learning
Final Project Report

Saptarishi Dhanuka, Divij Khaitan

April 12, 2026

Contents

1	Introduction	2
2	Literature Review	4
3	Data	5
4	Methods & Experimentation	6
4.1	Agents	6
4.2	Algorithms	7
4.3	Rewards	10
5	Results	11
5.1	Monte Carlo Tree Search	11
5.2	Meta Learner for Multi-Wordle	12
6	Conclusion	14

1 Introduction

The game we chose to solve in the midterm project was the popular [Wordle](#). The goal of the game is to guess a mystery five-letter word in six guesses, based on limited feedback. The rules of the feedback are:

1. Any letter in the guess placed in the same position as it would be in the target is marked in green
2. Any letter present in the target that is incorrectly placed in the guess is marked in yellow
3. In the case of duplicate misplaced letters, the first n occurrences of the letter in the guess will be marked yellow if the letter appears n times in the target
4. Any letters absent from the word or excess duplicate letters are marked in black/grey

There is a list of about 2000 words that the game chooses a target from daily. The goal of the project is to train an agent to win consistently and quickly at this game, using reinforcement learning techniques.

As we talk about in following sections, we first tried training agents from scratch to predict the best possible word, but those did not give good results at all due to the large search space of possible words. Hence we switched to a meta-learning approach with "heuristic agents", which instead learnt to choose from a set of deterministic heuristics, which gave us much better performance.

While we used different environments while formalizing the problem due to our different approaches, it can be broadly described as an MDP with the following components.

- **State Space:** Every possible state of the board i.e. the combination of the guesses made alongside the feedback, modulo permutations of letters and guess orders. To ensure a consistent ordering, the guesses may be sorted in alphabetical order. Moreover, the Markovian property comes from the fact that each state encodes all the words that have been guessed so far and the feedback that was received from each guess. This state space was used for training agents/algorithms from scratch for trying to predict the next word, which proved to be infeasible. The state space for the meta-learning approach was the feedback received at each step in terms of the number of greens, yellows and greys.
- **Action Space:** For the from-scratch agents, the actions were the next word to predict, while for the meta-learners it was which heuristic to pick.
- **Rewards:** The most basic reward structure is to reward the agent +10 for winning the game and -10 for losing. We experimented with a variety of reward structures that included intermediate rewards and penalties for greens, yellows and greys, which we explain in section 4.
- **Transition Probabilities:** The transitions for this game are completely deterministic, moving from a board state with k guesses made to one with $k + 1$ guesses made, with the feedback being decided by the rules outlined above.
- **Gamma:** A goal of the game is speed alongside correctness, gamma should be set to less than 1.

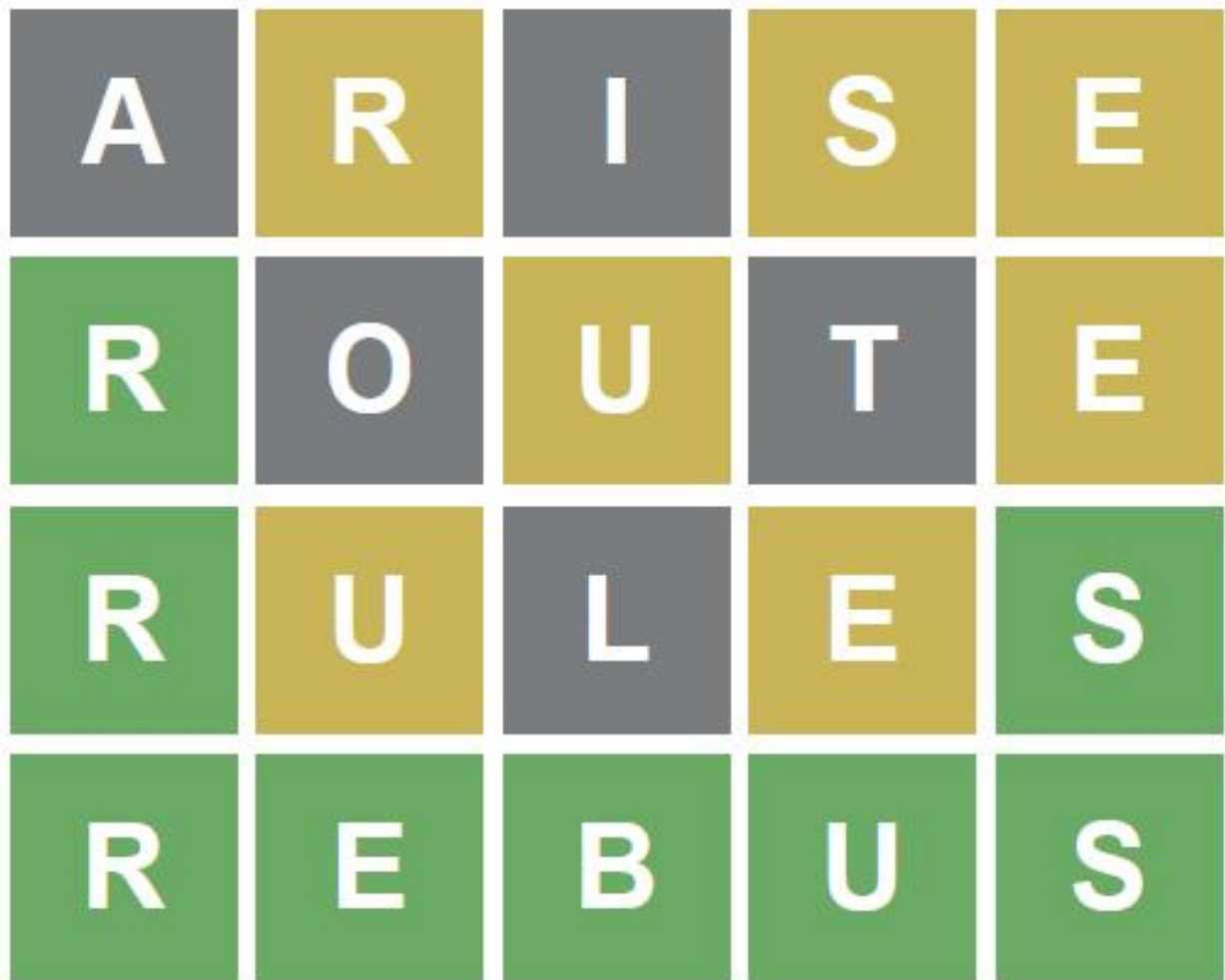


Figure 1: Example game of Wordle

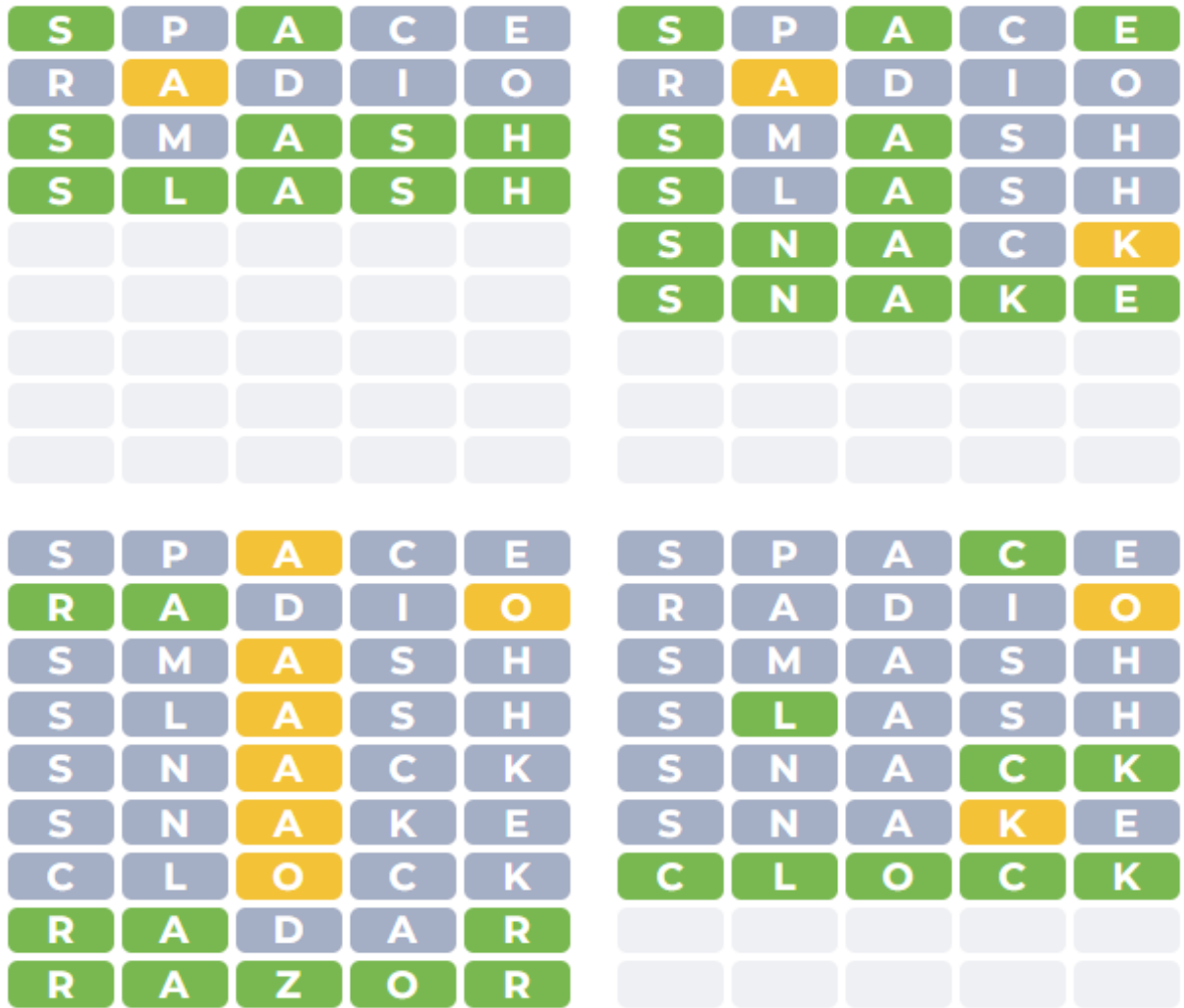


Figure 2: Example game of Wordle Variant with four boards

For the final project, we implemented Monte Carlo Tree Search for the regular wordle, as well as tackled a more difficult variant of the game with multiple boards and a longer length for allowed guesses. In this, two different Wordle Boards are active at the same time, and any guess entered is made across all four boards with ten total guesses. The goal of the agent is to win on both the boards as fast as possible.

2 Literature Review

Bhambri, Bhattacharjee and Bertsekas [4] use RL to attempt to improve upon a base heuristic, eliminating all impossible words from consideration as the guesses are rolled out. This is done by computing the word which has the least average distance from the entire remaining list under the heuristic. Sanderson [1] uses information theory to maximise the gain of information on each subsequent guess based on the entropy of words in the list of target words. Anderson [2] trains an agent to select the best heuristic for every situation from a set of heuristics. Short [6] guesses the word with the maximum posterior probability based on all known information upto that point.

3 Data

The word list for wordle can be found on [3]. The game allows around 14000 valid words to be guessed, however only 2309 of the words will actually be chosen to be the target. Our experiments primarily focus on the set of 2309 words due to computational constraints, while a few of our experiments include the full set of 14000 words.

We didn't carry out any extensive data exploration and analysis since our base dataset was a simple enough list of words. We also didn't want to extract statistics from the dataset and use an information theoretic approach, since that would deterministically brute force the problem without using any reinforcement learning.

4 Methods & Experimentation

This will be broken down into 3 sections: Agents, Algorithms, and Reward Structures

4.1 Agents

4.1.1 Meta-Learner Agent

As with the single wordle, we were facing difficulties with trying to make agents and algorithms learn from scratch due to the large search space of the problem and the variety of different strategies that agents could possibly learn, so we decide to use a meta-learning strategy for the multi-wordle scenario as well.

Here the agent has access to a number of different deterministic heuristics and it has to learn to pick the best heuristic instead of learning from scratch what word to predict next. Loosely, heuristics can be defined as simplified rules of thumb that may not always be applicable and no formal guarantees can be made about. Examples from chess would be rules taught to beginners such as not moving the same piece multiple times in the opening, controlling the opponent's access to the central squares and not pushing pawns in front of a castled king.

The agent's actions were to choose a particular heuristic. After reducing the search space through the heuristics, we then just picked words randomly from this smaller search space, which gave surprisingly good performance as shown in the results. For the multi-wordle setup we had to make some changes to the environment.

We tried out various approaches that tried to make a complicated environment for the meta-learner by taking into account the feedback from multiple different wordle environments. But none of them got anywhere close to good performance for the Q learner. So hence we essentially tried to wrap two separate single wordle environments into one environment. Then in the `step` method of the environment, incorporated feedback from one of the two environments at random. Based on this feedback, the agent chooses which heuristic to pick. The heuristics include

1. **Random:** Choosing a word at random
2. **Without absence:** Ensuring that any letter-position pairs known to be impossible (black feedback) are missing from the next action by filtering through words appropriately
3. **Confirmed without absence:** The same as the above, with the additional stipulation that any confirmed letters (green feedback) must be at the correct position in the next action
4. **Misplaced and Confirmed without absence:** Ensuring that missing letters are absent, confirmed letters are present and misplaced letters (yellow feedback) are incorporated in accordance with their counts. With regards to the misplaced/yellow letters, it kept only words that contain every yellow letter somewhere.
5. **Letter frequency:** By finding empirical distribution of letters in the English language, we score each word in the target word list by counting how many common English letters it contains (plus a small bonus for overall uniqueness).

6. **Green, Yellow and Grey Information Incorporated:** This was the most involved heuristic where it was mostly similar to heuristic 4, but with the added constraint that we ban yellow letters from the specific indices where they were previously guessed.

This is in the file `my_multi_wordle_env_2.py`.

4.2 Algorithms

4.2.1 AlphaZero, Monte Carlo Search and Upper Confidence Trees

We attempted to re-implement the AlphaZero methodology for wordle. This has four ingredients,

- Monte Carlo Game Tree Search
- UCT Algorithm
- Neural Networks for Value and Policy Approximation
- Experience Replay Buffer

Game trees are a representation of every possible outcome of every possible decision of each player at every step of a sequential game. An example can be found in Figure 3.

Exploring these trees to find the optimal decisions have spawned many algorithms such as minimax and alpha-beta search, among them being monte carlo tree search. As the name suggests, this relies on a random sampling of paths on the tree to estimate the best decision. The algorithm has 4 steps

1. Selection: Starting from some root node R , children $C_1 \dots C_n$ at levels $1, \dots, n$ are chosen until a leaf node L is reached. A leaf is any node from which no further simulations have been made.
2. Expansion: If L ends the game, the search ends. If not, a child of L is chosen at random and the game tree is expanded. Let the child be denoted by K .
3. Simulation: Now, a game is simulated starting from the sub-tree rooted at K . This is often also called the rollout or playout stage. The outcome of the game is recorded.
4. Backpropagation: Along the path from K to R , the information about the outcome of the game is distributed through backpropagation.

The 4 steps for the algorithm are illustrated in Figure 4.

Upper Confidence bounds for trees (UCT) was proposed by Kocsis and Szepesvári in [5]. These are an extension of the UCB1 algorithm, commonly used for the multi-armed bandit problem to Monte Carlo Tree searches. The formula for UCT is

$$a^* = \operatorname{argmax}_a Q(s, a) + c \sqrt{\frac{\log(N_s)}{1 + N_s}}$$

. We use a variant of this algorithm called PUCT, adapted from PUCB which adds in a prior belief about the probability of winning from a particular node

$$a^* = \operatorname{argmax}_a Q(s, a) + P(s, a) c \sqrt{\frac{\log(N_s)}{1 + N_s}}$$

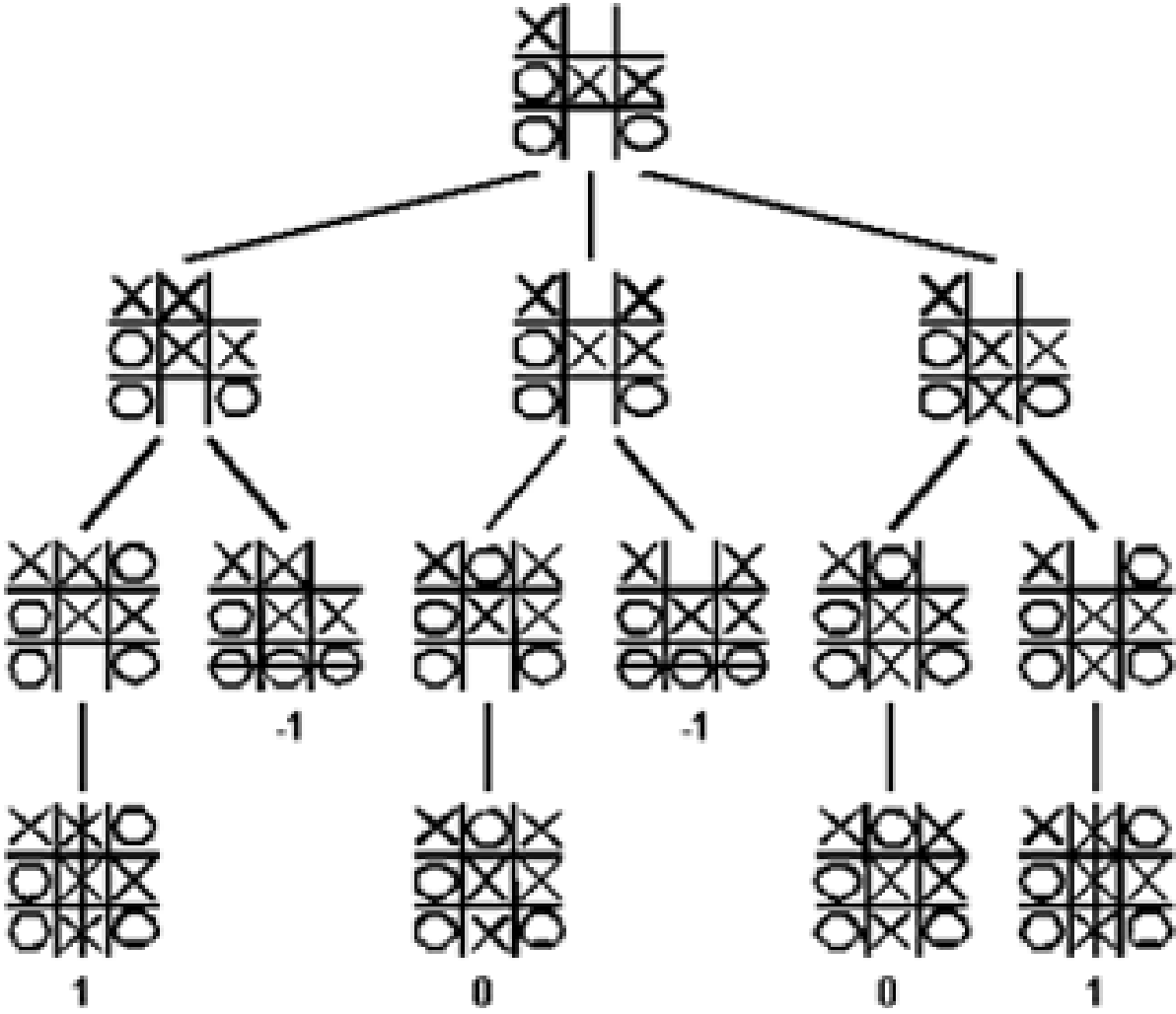


Figure 3: Example of a Game Tree for Tic Tac Toe

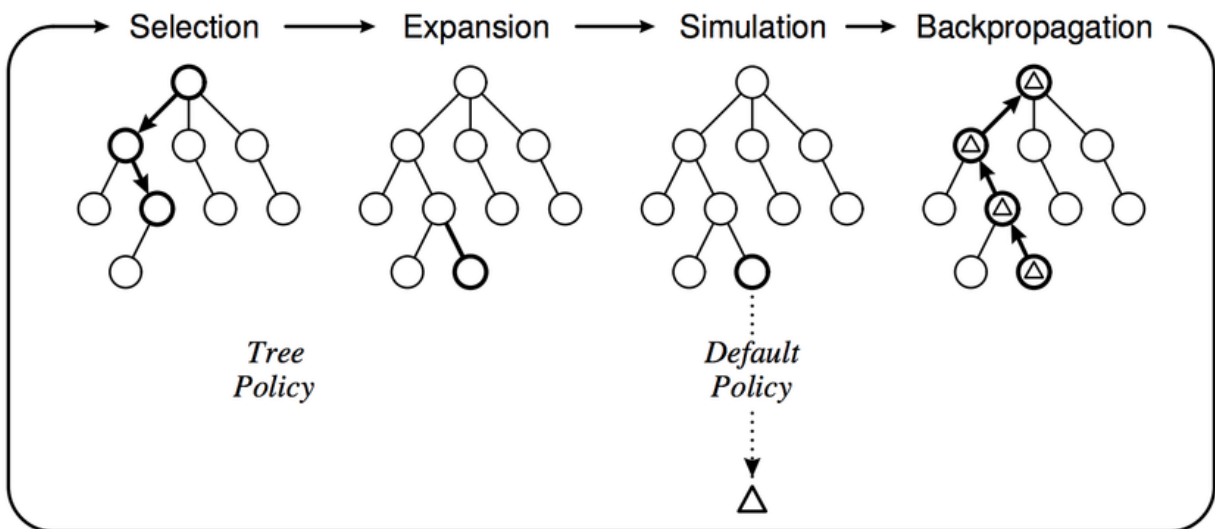


Figure 4: Illustration of the flow of Monte Carlo Tree Search

In both formulae Q is the fraction of runs which resulted in a win, N_s is the number of visits to the state, c is called the exploration constant and $P(s, a)$ is the prior belief. c was proven to be $\sqrt{2}$ theoretically, but is often tuned in practice.

Alphazero was revolutionary not just because of its scale, but also because of its use of deep learning in the tree search procedure. AlphaZero makes use of two neural networks one for the value function and one for the policy function. When this network is added, the loop becomes

- Selection: proceeds as normal as per the policy priors, usually uniform at initialisation
- Expansion: The neural network is used to initialise the estimates for the policy prior the value of the child node
- Simulation: The value of the leaf node can now be calculated from the neural network without needing to simulate the entire trajectory
- Backpropagation: Proceeds as normal alongside the path

The loss function for this network is

$$L(\theta) = (z - V(\theta))^2 - \langle \pi(\theta), P(s) \rangle$$

Often an $L2$ weight decay term is added, but this was omitted during our training. The value target comes from the outcome of the game, while the policy target comes from the number of times a state was visited in proportion to the total number of visits made to all children of that parent.

4.2.2 Q-Learning

The goal of this is to estimate a table $Q(s, a)$, where the Q -function is the expected reward of taking action a while in state s . Since wordle is a finite horizon problem of at most six steps, the table has an additional index h for the number of steps. The table is updated using the Bellman equations.

$$Q_h(s, a) := Q_h(s, a) + \alpha(\mathbb{E}_{s'}[r + \gamma(\max_{a'} Q_{h+1}(s', a') - Q_h(s, a)) | (s, a)])$$

The table is initialised with all of the entries having value 0. Ties between actions are broken arbitrarily, and an ϵ -greedy strategy is used to balance exploration with exploitation. [7]

For the from-scratch agent we didn't get any good performance due to the explosion in the state space. The training reward also barely showed any improvement.

For the heuristic agent, we use a form of Temporal Difference (TD) Learning where the exact update rule is given as below

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

where the states are the greens, yellows and greys and the actions are which heuristic to pick.

4.3 Rewards

It is conventional to only reward the agent upon the completion of the game, however wordle lends itself to intermediate feedback and hence intermediate rewards. We tried assigning rewards based on the feedback, such as a +10 for a green, -1 for a black, + 2 for a yellow etc.. The other version we tried was the conventional method of rewarding the agent only upon completion, penalising losses and reinforcing wins. For the multi wordle scenario, we just added the rewards from each environment.

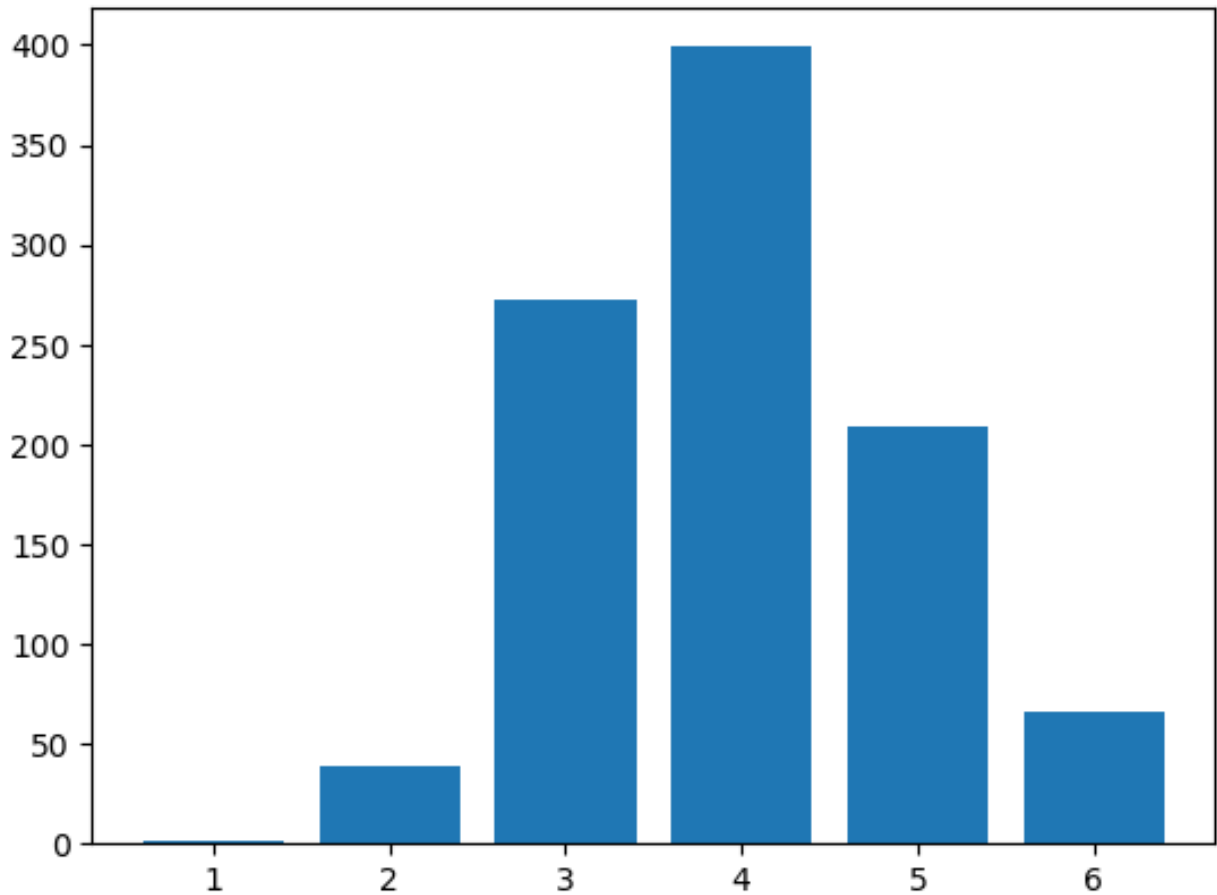


Figure 5: Results for Vanilla Monte Carlo Tree Search

5 Results

We focus on the MCTS for single wordle and the meta-learner for the multi-wordle scenario since the latter was the only one for which we got any good results

5.1 Monte Carlo Tree Search

The results for the vanilla monte carlo tree search are attached in Figure 5. The accuracy of the method is 98.70%, with an average guess length of 4. While good, this clearly warrants improvement. At the time of the report submission, the AlphaZero Network has yet to converge. Due to computational constraints, the model was unable to finish training and was yet to register a single win. The results will be part of the presentation.

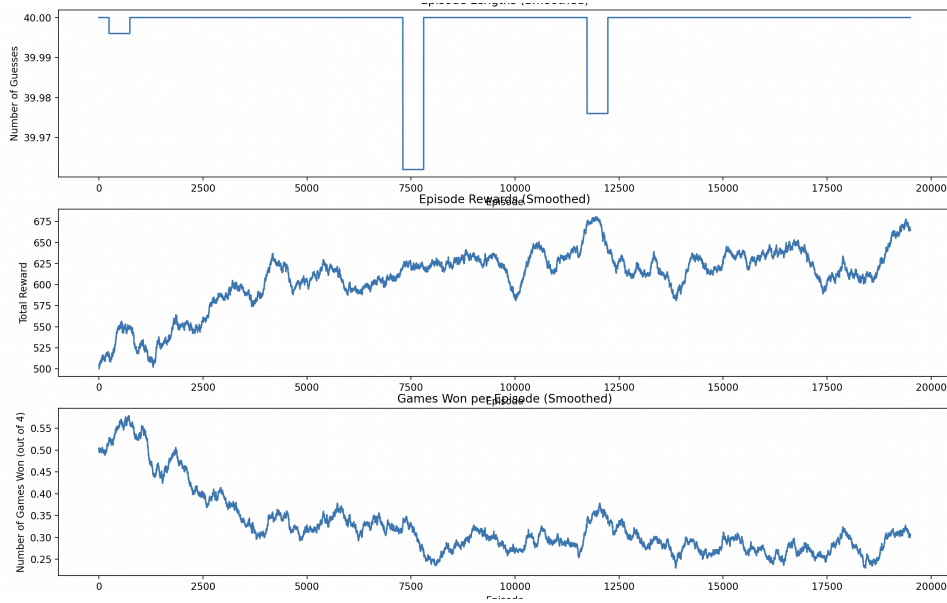


Figure 6: Bad performance for complicated multi wordle env

5.2 Meta Learner for Multi-Wordle

With the complicated environment we didn't get a good win rate at all as shown in Figure 6.

For the simpler environment the training curves for the number of games, reward and win rates are given in Figure 8.

For the simpler environment we could get an average game length of 7.65 guesses, which is close to our regular human performance, based on the times we have played the game.

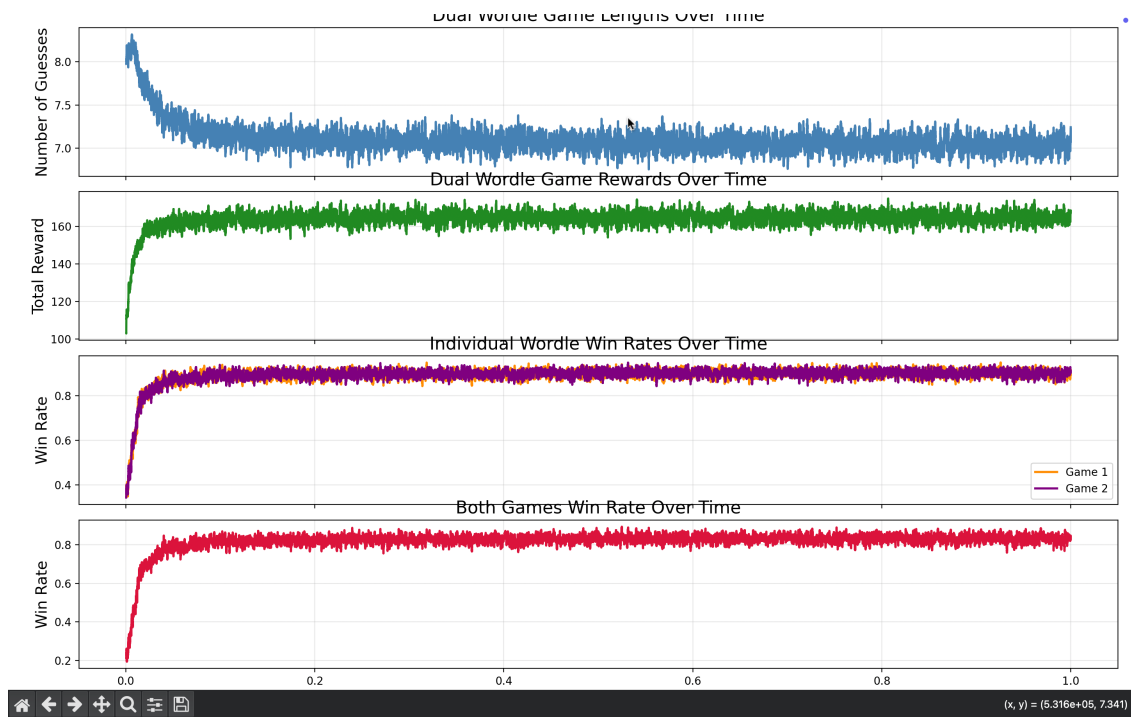


Figure 7: Training graphs for simpler multi-wordle

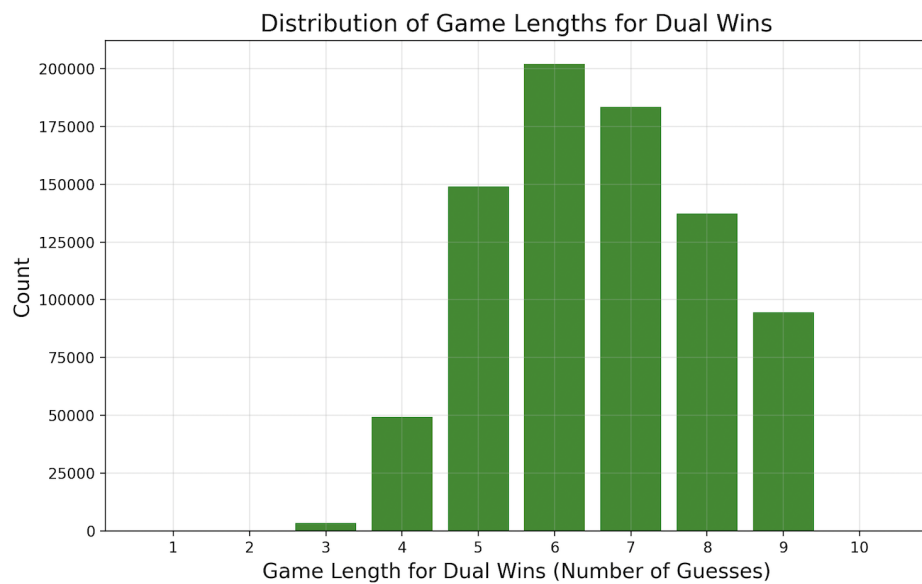


Figure 8: Histogram of game lengths

6 Conclusion

For the multi-wordle environment, we again spent a lot of time formalising the environment by building on top of the environment made for the single wordle variant. But when applying Q learning to a complicated environment that took into account the feedback from both the environments did not give good performance, with win rates nearing 0. So instead we decided to wrap two of our original environments into one environment, and then get feedback 50-50 at random from each environment. We also tried applying PPO and DQN to this setup, which similar to the midterm project did not give good performance.

For the Monte Carlo Tree Search approach, vanilla MCTS gave us great results, which suggests that a converged alphazero may have even better results than our meta-learner since it is a model based method rather than a model free method.

References

- [1] 3Blue1Brown. Solving wordle using information theory, 2022. Accessed: 2025-03-21.
- [2] Benton J. Anderson and Jesse G. Meyer. Finding the optimal human strategy for wordle using maximum correct letter probabilities and reinforcement learning, 2022.
- [3] Arjun Vikas (arjvik). Wordle wordlist, 2025. Accessed: 2025-03-21.
- [4] Siddhant Bhambri, Amrita Bhattacharjee, and Dimitri Bertsekas. Reinforcement learning methods for wordle: A pomdp/adaptive control approach, 2022.
- [5] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou, editors, *Machine Learning: ECML 2006*, pages 282–293, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
- [6] Martin B. Short. Winning wordle wisely, 2022.
- [7] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3):279–292, May 1992.